

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 05 -

GERENCIAMENTO DE DEPENDÊNCIAS

SUMÁRIO

O QUE VEM POR AÍ?3

HANDS ON4

SAIBA MAIS.....6

MERCADO, CASES E TENDÊNCIAS17

O QUE VOCÊ VIU NESTA AULA?19

REFERÊNCIAS.....20



O QUE VEM POR AÍ?

Você já se deparou com um projeto de ciência de dados que “funcionava”, mas falhou ao rodar em outro lugar ou atualizar alguma biblioteca? Por exemplo: “Com pandas 1.3 tudo estava ok, mas ao atualizar para 1.5 começou a dar erro!”. Esse tipo de problema é sintoma do “inferno das dependências” – quando bibliotecas incompatíveis ou versões conflitantes quebram seu código. Nesta aula, vamos explorar como evitar esses transtornos por meio do gerenciamento de dependências.

Você aprenderá a isolar o ambiente de cada projeto (usando ambientes virtuais), a documentar e fixar as versões necessárias (garantindo reprodutibilidade) e a empacotar código reutilizável de forma profissional. Tudo isso proporciona um desenvolvimento mais confiável e colaborativo: as análises tornam-se reproduzíveis e compartilhar seu projeto com o time ou implantá-lo em produção deixa de ser um pesadelo.

Preparado(a)? Vamos “blindar” seu projeto contra conflitos de bibliotecas e garantir que ele rode hoje, amanhã ou em qualquer lugar, sempre da mesma forma.

HANDS ON

Nestas videoaulas, vamos colocar em prática esses conceitos montando um ambiente isolado e gerenciando dependências com uma ferramenta moderna chamada UV. O UV é um gerenciador de pacotes Python de última geração, projetado para simplificar a vida do indivíduo desenvolvedor: ele combina as funções do pip, virtualenv, pipenv e outras ferramentas em um só utilitário rápido e eficiente.

Assim, você verá passo a passo como criar um novo projeto de data science usando o UV, adicionar bibliotecas populares (por exemplo, NumPy, pandas etc.) e gerar um lockfile que fixa as versões exatas dos pacotes. A ideia é mostrar, de forma prática, como evitar conflitos de versão e garantir que todos – você, seus colegas ou o servidor de produção – utilizem o mesmo ambiente configurado.

Dica: fique de olho nos comandos e saídas apresentados no vídeo e tente reproduzi-los por conta própria depois – essa é uma ótima forma de fixar o conhecimento! Vamos à demonstração:

```
# Criando um novo projeto Python e entrando no diretório
$ uv init meu_projeto_ds
Initialized project meu_projeto_ds at ./meu_projeto_ds
$ cd meu_projeto_ds

# Adicionando dependências ao projeto (ex.: pandas e numpy)
$ uv add pandas numpy
Creating virtual environment at: .venv
Resolved 5 packages in 2.1s
Installed 5 packages in 0.5s
+ pandas 1.5.3
+ numpy 1.21.6

# Executando um código dentro do ambiente para verificar
$ uv run python -c "import pandas; print(pandas.__version__)"
1.5.3
```

Código-fonte 1 – Demonstração de uso do UV para criar um ambiente virtual e gerenciar dependências de um projeto Python
Fonte: Elaborado pelo autor (2025)

No exemplo anterior, o UV inicializa um projeto com ambiente virtual automático e instala as bibliotecas especificadas com suas versões compatíveis. Você verá na videoaula que o UV também gera um arquivo pyproject.toml e um uv.lock (lockfile) com as versões exatas – garantindo que, se outro desenvolvedor ou desenvolvedora rodar os mesmos comandos, terá o mesmo ambiente.

Aproveite as videoaulas para se familiarizar com essa ferramenta – ela representa uma tendência de simplificação do workflow em Python (mais detalhes na próxima seção!). Após assistir, volte aqui para aprofundarmos cada conceito que foi aplicado.



SAIBA MAIS

Agora que você acompanhou a configuração de um ambiente no hands on (se ainda não assistiu, vale a pena conferir os vídeos antes de prosseguir!), vamos mergulhar nos princípios e melhores práticas de gerenciamento de dependências em projetos de ciência de dados. Nesta seção, discutiremos em detalhes por que e como isolar dependências por projeto, como documentar e fixar requisitos de forma sólida (incluindo noções de versionamento semântico) e como empacotar código reutilizável para compartilhamento.

Tudo isso em uma linguagem acessível, porém tecnicamente rica – com direito a pitadas de matemática computacional, notas sobre desempenho (CPU/GPU) e referências teóricas de ciência da computação pertinentes. Vamos construir esses conceitos de forma leve e progressiva, conectando teoria e prática para consolidar seu aprendizado.

Isolamento de dependências: ambientes virtuais

O primeiro passo para evitar caos nas dependências é isolar o ambiente de cada projeto. Em vez de instalar pacotes globalmente no sistema (o que pode causar conflitos terríveis entre projetos diferentes), a prática recomendada é usar ambientes virtuais – “bolhas” separadas que contêm uma instalação do Python e dos pacotes necessários só daquele projeto. Como vimos, se instalarmos tudo globalmente, basta que um projeto exija NumPy 1.19 enquanto outro precise de NumPy 1.25 para estarmos encrencados.

Já com ambientes virtuais, cada projeto “vive” em sua bolha isolada, podendo ter versões distintas da mesma biblioteca sem brigar umas com as outras. Isso traz estabilidade e paz de espírito: o que você instalar dentro do ambiente do Projeto A não afeta o Projeto B.

No ecossistema Python, há várias formas de criar ambientes virtuais. A ferramenta mais básica é o módulo integrado venv (ou o predecessor virtualenv). Com ela, você pode fazer: `python3 -m venv .env` para criar um ambiente (pasta) chamado `.env`, e depois ativá-lo (`source .env/bin/activate` no Linux/macOS). Quando

ativado, qualquer pip install que você rodar instalará os pacotes dentro desse ambiente, não no sistema global.

Assim, você consegue, por exemplo, ter um projeto rodando com pandas 1.3 e outro com pandas 2.0 no mesmo computador, sem conflitos – cada um no seu canto. Ao terminar, é só rodar deactivate e pronto: você sai do ambiente virtual, retornando ao Python global. Essa separação por projeto evita o “vazamento de esgoto” entre sistemas, como brinca Juha Kiili: “não deixe o esgoto vazar, use ambientes virtuais!”.

Outra abordagem muito usada por cientistas de dados é o Conda, que combina gerenciamento de pacotes e ambientes. O Conda permite criar ambientes (`conda create -n meu_env python=3.9 pandas=1.5 etc.`) e tem a vantagem de gerenciar não apenas pacotes Python, mas também dependências nativas/SO (por exemplo, bibliotecas C, MKL, CUDA para GPUs etc.). Isso o torna popular para setups de Machine Learning, em que pacotes como TensorFlow ou PyTorch às vezes precisam de binários específicos.

Em resumo: ambientes virtuais são essenciais para garantir reprodutibilidade e evitar conflitos. Eles isolam versões de bibliotecas e até mesmo versões do interpretador Python. Sem isso, é fácil cair no inferno de dependências em projetos grandes. Atualmente, seja com venv, Conda, Poetry (que também cria ambientes automaticamente) ou mesmo ferramentas como o UV que vimos – o importante é nunca mais “instalar tudo globalmente”.

Antes de prosseguir, um insight de desempenho: ambientes virtuais também ajudam a controlar o consumo de recursos. Como cada ambiente carrega só o necessário, você evita inflar processos Python com bibliotecas não usadas. Em um experimento rápido, executar um script simples em um ambiente “enxuto” inicia o processo Python usando menos memória do que em um ambiente global atolado de libs. Ou seja, isolamento não é só questão de organização – pode trazer leve ganho de eficiência (embora sutil) ao não carregar módulos desnecessários.

Além disso, para projetos que eventualmente serão paralelizados ou distribuídos (por exemplo, rodar instâncias simultâneas em um cluster), ter ambientes autônomos facilita replicar a configuração exata em múltiplos nós,

garantindo que cada um use versões idênticas de pacotes e evitando discrepâncias que poderiam causar falhas difíceis de depurar.

Documentação e fixação de requisitos (requirements)

Tão importante quanto isolar o ambiente é documentar as dependências do projeto. Isso normalmente é feito listando os pacotes e versões em um arquivo de requisitos, como `requirements.txt` (para pip) ou `environment.yml` (para Conda). A ideia é simples: em vez de dizer para alguém "instale numpy, pandas, etc.", você fornece uma lista explícita do que instalar. Assim, qualquer pessoa (ou servidor CI/CD) pode recriar o ambiente rodando um simples `pip install -r requirements.txt` ou `conda env create -f environment.yml`.

Um benefício enorme disso é na onboarding de novos membros: todo(a) cientista de dados já perdeu horas tentando rodar o projeto de um(a) colega porque "faltava instalar X ou Y". Com um bom arquivo de requisitos, esse tempo se reduz – os builds ficam estáveis entre máquinas: "se funciona na minha máquina, vai funcionar na sua".

Mas não basta listar os pacotes – é crucial fixar versões. Lembra do nosso exemplo do Pandas? Se no seu `requirements.txt` estiver apenas Pandas (sem versão), hoje ele instalará a versão mais nova (digamos 1.5.3), mas daqui a alguns meses pode ser 1.6 ou 2.0 e seu código talvez quebre. Isso significa que um `requirements.txt` sem versões não garante reprodutibilidade temporal: uma instalação hoje pode ser diferente de uma instalação amanhã.

A solução é usar versionamento semântico (SemVer) e pinagem: por exemplo, `pandas==1.5.3` especifica a versão exata. Assim, você "congela" o ambiente no tempo – o que é importante para projetos de longo prazo e publicações científicas, em que você quer conseguir rodar o código anos depois e obter os mesmos resultados.

A prática recomendada é listar todas as dependências com versões exatas (incluindo dependências transitivas, aquelas que seu projeto não importa diretamente, mas vêm "de brinde" com outras). Ferramentas como pip freeze ajudam

a gerar esse snapshot. No Conda, o `environment.yml` pode ser criado com `conda env export`, incluindo todos os pacotes e versões.

Vamos falar um pouco sobre versionamento semântico, já que fixar versões envolve entender de versões. O SemVer define que versões são identificadas por três números: MAJOR.MINOR.PATCH, por exemplo 2.4.1. Quando apenas o PATCH muda (ex: 2.4.1 -> 2.4.2), é uma correção de bug sem alterar interface; MINOR (2.4 -> 2.5) indica funcionalidades novas, mas compatíveis; MAJOR (2.x -> 3.0) indica mudanças incompatíveis (breaking changes). Por que isso importa?

Porque ao definir requisitos, você pode flexibilizar algumas atualizações. Por exemplo, você pode querer “qualquer versão da série 1.x do pacote X”. Nas notações do pip, `X>=1.0,<2.0` significa aceite 1.anything. Isso aproveita atualizações de bug e pequenas features sem quebrar compatibilidade. Mas cuidado: confiar que todo pacote segue SemVer à risca pode ser arriscado.

Muitas equipes optam pelo caminho mais seguro: pinam tudo exatamente, e quando desejam atualizar algo, fazem isso manualmente, testam o projeto e então atualizam o arquivo de requisitos. Essa instalação determinística (sempre os mesmos bits) previne surpresas desagradáveis.

Uma técnica útil aqui é diferenciar dependências diretas e indiretas. Ferramentas como pip-tools (pip-compile) permitem que você mantenha um arquivo de “requisitos de alto nível” (por exemplo, `requirements.in` contendo apenas `pandas==1.2.1` e `matplotlib==3.4.0` que você usa diretamente) e então gerem automaticamente um `requirements.txt` completo com todas as subdependências pinadas (numpy, pytz, etc).

Veja um exemplo resumido do Valohai blog: você especifica `matplotlib==3.4.0` e `pandas==1.2.1` e a ferramenta resolve e produz um `requirements.txt` listando `numpy==1.22.0`, `python-dateutil==2.8.2` etc., assegurando uma instalação consistente. Esse workflow ajuda a manter seu arquivo de requisitos organizado e confiável. Em projetos complexos, vale também adotar avaliação contínua das versões: agendar para, a cada certo período, atualizar as libs (por exemplo, subir da versão 1.2 para 1.3) e rodar todos os testes.

Isso evita acumular defasagens muito grandes – o que pode virar um problema de manutenção depois. Aliás, muitas empresas utilizam bots como o

Dependabot (do GitHub) que automaticamente sugerem PRs de atualização de dependências, já sinalizando mudanças de versão (MAJOR/MINOR) e até possíveis riscos.

Resumindo esta parte: mantenha sempre um inventário das dependências do seu projeto. “Não dependa da memória ou de comentários soltos no código para dizer que precisa do pacote X”. Use arquivos padronizados e inclua-os no controle de versão (Git). Assim, qualquer um, em qualquer lugar, pode recriar o seu ambiente. E lembre-se: versões importam! Documente-as.

Essa disciplina é parte da maturidade profissional que aproxima o(a) cientista de dados de boas práticas de engenharia de software. Projetos reprodutíveis economizam horas de debug e mostram cuidado com a qualidade – algo muito valorizado em contextos colaborativos e de produção.

Ferramentas modernas: Poetry, Pipenv, UV e outros gerenciadores

Gerenciar dependências “na mão” com pip + requirements.txt funciona, mas pode ser trabalhoso à medida que o projeto cresce. Nos últimos anos surgiram ferramentas para facilitar essa tarefa. Duas das mais conhecidas são o Pipenv e o Poetry e ambas procuram tornar o fluxo mais simples e confiável, integrando criação de ambiente virtual, instalação de pacotes e geração de lockfiles automaticamente.

No Poetry, por exemplo, você declara as dependências em um arquivo pyproject.toml (ex.: `numpy = "^1.21.0"`, que significa “versão 1.21.x mais recente”) e ele resolve as versões compatíveis, instala em um ambiente isolado e gera o poetry.lock com as versões exatas escolhidas. Na próxima vez, o poetry install recria exatamente aquele estado travado no lockfile.

O Poetry também simplifica a publicação de pacotes, caso seu projeto evolua para uma biblioteca reutilizável, coisa que o Pipenv não faz. Já o Pipenv introduz os arquivos Pipfile e Pipfile.lock, cumprindo um papel similar ao Poetry, embora hoje o Poetry tenha se tornado mais popular e atualizado.

E o UV que usamos no hands on? O UV é uma novidade (surgiu por volta de 2024) que promete ser um gerenciador “tudo em um”: ele substitui pip, virtualenv, pipenv/poetry, pyenv e até o build de pacotes, combinando todas essas funções com desempenho elevado (escrito em Rust). Com um único comando `uv add package`,

ele cria o ambiente virtual se ainda não existir, instala o pacote com dependências resolvidas e atualiza seu `pyproject.toml` e `uv.lock`.

As vantagens incluem velocidade (instalações até 10× mais rápidas que pip tradicional) e resolver moderno de dependências que evita muitos conflitos. Além disso, o UV suporta workspaces (projetos múltiplos com lockfile universal) e gestão de múltiplas versões de Python, tornando-o atraente para uso profissional. Na prática, o UV e o Poetry têm objetivos bem alinhados – ambos produzem lockfiles, gerenciam ambientes virtuais e facilitam publicação – mas o UV destaca-se pela performance e por abranger ferramentas legadas (é compatível com `requirements.txt`, por exemplo).

A tabela a seguir resume as características de algumas dessas ferramentas:

CARACTERÍSTICA	PIP + VIRTUALENV	CONDA	POETRY	UV (ASTRAL)
Ambiente virtual	Necessita criar/ativar manual (venv)	Integrado (conda env)	Integrado (auto-virtualenv)	Integrado (auto)
Gerência de pacotes	pip install (não gera lock)	conda install (pacotes Py/Não-Py)	poetry add (gera poetry.lock)	uv add (gera uv.lock)
Velocidade	Base (serial)	Moderada (depende do caso)	Boa (resolver em Python)	Muito alta (paralelo em Rust)
Lockfile	Manual (pip freeze > req.txt)	environment.yml (pode fixar)	Sim (poetry.lock)	Sim (uv.lock)

Não-Python (ex.: C libs)	Não gerencia	Sim (conda baixa libs nativos)	Não (usa pip internamente)	Não (foca só em pacotes Py)
Publicar pacote	Via setup.py/twine (manual)	N/A	Sim (poetry publish)	Sim (uv publish)

Tabela 1 – Comparativo entre ferramentas de gerenciamento de dependências e ambientes em Python

Fonte: Elaborado pelo autor (2025)

Como podemos ver, as ferramentas modernas como Poetry e UV trazem conveniências (automatizam criação de ambiente e lockfiles) e reforçam boas práticas (sempre gerando lockfiles para builds determinísticos). Já o Conda permanece indispensável em cenários que envolvem dependências de baixo nível ou múltiplas linguagens – muitos projetos de ciência de dados usam uma combinação, por exemplo: criar um ambiente Conda e dentro dele usar Poetry para gerenciar os pacotes Python. O importante é reconhecer as fortalezas e limitações de cada abordagem.

Em termos de ciência da computação, esses gerenciadores implementam soluções para o clássico problema de satisfação de dependências (resolver versões compatíveis que atendam a todas as restrições). Algoritmos de dependency resolution vêm evoluindo – o pip, por exemplo, adotou um resolvedor mais inteligente em 2020 para lidar melhor com conflitos circulares e backtracking, inspirando-se em algoritmos de satisfação booleana (SAT).

Ferramentas como Poetry e UV foram além, usando estruturas de resolução modernas que garantem encontrar uma solução (ou falhar claramente) mais rapidamente, muitas vezes empregando múltiplos threads (no caso do UV) para baixar e analisar metadados dos pacotes. Isso reflete a preocupação em otimizar o tempo de setup: afinal, quanto menos tempo gastamos configurando ambiente, mais sobra para analisar dados e treinar modelos!

Em relação a controle de recursos, vale notar que o gerenciador de pacotes em si pode afetar a performance do ciclo de desenvolvimento. Por exemplo, o UV promete usar menos memória durante a instalação, graças à implementação em Rust, o que é benéfico quando se tem projetos enormes com centenas de dependências.

Não é incomum pip ou conda consumirem bastante RAM ao resolver complexas árvores de dependência – otimizações nesse processo são bem-vindas principalmente em CI (Integração Contínua), em que máquinas de build podem ser modestas. Assim, a escolha da ferramenta de gerenciamento também pode influenciar produtividade e eficiência do time de ciência de dados em escala.

Antes de fechar este tópico, mencionemos uma extensão importante do gerenciamento de dependências: quando falamos de modelos de Machine Learning e pipelines de dados, as dependências vão além de pacotes de código – incluem dados treinados, hiperparâmetros etc.

Surgiram ferramentas como o MLflow que, entre outras coisas, permitem empacotar modelos junto com uma “snapshot” de ambiente (por exemplo, ele gera um requirements.txt ou conda.yaml dentro do artefato do modelo). Isso garante que ao carregar um modelo treinado, você saiba exatamente com quais libs ele foi criado.

Embora não entremos em detalhes de MLflow aqui, é bom saber que o conceito de dependências se estende também a pipelines e modelos, e ferramentas de MLOps tratam de versionar e gerenciar isso para a reprodutibilidade de experimentos. É mais um exemplo de aplicação do que aprendemos: seja código ou modelo, registrar com o que ele foi feito é fundamental para poder refazer igual no futuro.

Empacotamento de código reutilizável

Conforme seus projetos crescem, é provável que certas partes do código sejam úteis em múltiplos projetos. Em vez de copiar-e-colar esses utilitários em cada novo repositório (o que dificulta manutenção e fere princípios DRY – Don't Repeat Yourself), a abordagem de engenharia de software é empacotar o código reutilizável em uma biblioteca.

Isso significa transformar aqueles módulos e funções comuns em um pacote instalável (via pip ou Conda), interno ou público. Por exemplo, suponha que sua equipe desenvolveu funções ótimas de pré-processamento de dados que são utilizadas em vários notebooks. Melhor do que replicar o código em cada notebook seria criar um pacote Python, digamos `acme_preprocessing`, colocar essas funções lá e distribuí-lo internamente. Assim, cada projeto pode simplesmente importar esse pacote e você centraliza as melhorias nele.

Como fazer isso na prática? Entra em cena o conceito de empacotamento e distribuição de pacotes Python. Tradicionalmente, criamos um arquivo `setup.py` ou, mais recentemente, um `pyproject.toml` conforme o PEP 518/PEP 621, que descreve metadata do pacote (nome, versão, autor, dependências necessárias etc.). Você organiza seu código em uma estrutura de diretórios (por exemplo, todos os módulos dentro de `src/nome_do_pacote/` com um `__init__.py` vazio indicando que é um pacote).

Depois, com ferramentas como `setuptools` ou `hatch` ou o próprio Poetry, você pode construir um arquivo distribuível (um wheel `.whl` ou source distribution) e compartilhá-lo. Se for interno, talvez vocês tenham um índice privado ou até mesmo podem instalar diretamente via repositório Git. Se for aberto, pode publicar no PyPI (Python Package Index) para que qualquer pessoa instale via `pip install seu-pacote`.

O importante é que, ao empacotar, você declara explicitamente as dependências do pacote e outras restrições, facilitando a integração em outros projetos. Além disso, isso impõe modularização: normalmente um pacote bem feito tem responsabilidades claras, documentação e testes dedicados – elevando a qualidade geral do código.

Do ponto de vista de manutenção, imagine a situação: você identificou um bug em uma função reutilizada por 5 projetos. Se essa função foi copiada em 5 lugares, terá que corrigir nos 5, um a um. Se ela estiver em um pacote compartilhado, você corrige em um só lugar (o pacote), lança uma nova versão (ex.: 1.2.1 -> 1.2.2 no PATCH) e todos os projetos podem atualizar a versão do pacote para receber a correção.

Isso é muito mais escalável e segue princípios de reutilização de software. Empresas de tecnologia maduras incentivam isso – por exemplo, o Spotify, o Airbnb

e outras têm verdadeiros ecossistemas de pacotes internos, em que equipes publicam pacotes para uso umas das outras (desde funções utilitárias até modelos de ML versionados como pacotes). Essa prática reduz retrabalho e promove consistência nas soluções.

Empacotar também força você a pensar em design de APIs do seu código, já que outros irão usá-lo apenas via as funções/classes públicas que você expõe. É um ótimo exercício de engenharia: “como tornar meu código de data science fácil de consumir?”. E não precisa se assustar – para pacotes simples, um `pyproject.toml` mínimo e alguns comandos do Poetry ou do build do PyPA já geram seu wheel em minutos. Ferramentas como `cookiecutter` e `copier` têm até templates prontos que criam a estrutura básica do pacote para você começar.

Vamos a um exemplo hipotético: você tem um módulo com funções para cálculo de erro médio absoluto percentual (MAPE) e de previsão usando dois modelos comuns. Você decide empacotar isso como `minha_lib_ml`. Cria `minha_lib_ml/__init__.py`, `minha_lib_ml/metrics.py` (com a função `mean_absolute_percentage_error`) e `minha_lib_ml/prediction.py` (com funções de previsão). No `pyproject.toml`, define nome `minha-lib-ml`, versão `0.1.0`, dependências necessárias (por ex., `numpy >=1.21`) e aponta `setuptools/hatch` como backend de build.

Ao rodar `pip build` ou `poetry build`, gera-se `minha_lib_ml-0.1.0-py3-none-any.whl`. Você publica esse wheel (no PyPI ou repositório interno). Agora, em outro projeto, basta `pip install minha_lib_ml==0.1.0` e usar `from minha_lib_ml.metrics import mean_absolute_percentage_error`. Pronto – aquele outro projeto incorporou seu código sem você duplicar nada lá dentro. Futuramente, se melhorar a função MAPE (sem mudar a interface), pode lançar `0.1.1`; se adicionar novas funções, talvez `0.2.0`; se fizer mudança incompatível, `1.0.0` (aplicando SemVer). Os projetos que usam sua lib podem escolher se e quando atualizam para a nova versão, de acordo com a necessidade. Isso cria uma separação clara entre o núcleo reutilizável e as aplicações específicas.

Um cuidado: ao empacotar, preste atenção para não incluir dependências desnecessárias. Puxe apenas o que for realmente usado, para manter seu pacote leve. E evite pinagem muito rígida dentro de um pacote (diferente de aplicações). Por exemplo, se seu pacote interno usa `numpy`, pode ser melhor declarar

`numpy>=1.21,<1.25` do que `numpy==1.23.4`, para não forçar todos os projetos a aquela versão exata – deixe certa flexibilidade sem quebrar compatibilidade (pacotes reutilizáveis geralmente seguem essa filosofia, conforme notado no Valohai: bibliotecas tendem a não pinarem tudo rigidamente, deixando isso a cargo da aplicação final). Essa distinção é interessante: projetos/produtos querem determinismo (pin total), bibliotecas querem compatibilidade (pin parcial).

Empacotar código de ciência de dados também pode envolver publicar resultados científicos como pacotes, reforçando a reprodutibilidade acadêmica. Muitos artigos hoje trazem, além do código-fonte, um pacote instalável com as implementações das técnicas. Isso facilita que outros reproduzam experimentos ou apliquem métodos em seus dados. É uma convergência legal entre a engenharia de software e a ciência de dados: fornecer artefatos reutilizáveis além de gráficos e conclusões.

Para finalizar esta seção, vale sublinhar os ganhos: projetos modulados em pacotes têm manutenibilidade maior, menor redundância de código e promovem uma cultura de compartilhamento saudável (dentro e fora da organização). Ferramentas como GitHub + PyPI tornam trivial compartilhar um pacote publicamente e internamente podem ser usados registries privados (ou até mesmo importar direto via Git URL).

Se você nunca criou um pacote Python antes, tente fazer um experimento com algo simples do seu dia a dia – a experiência vai lhe trazer uma nova perspectiva sobre organizar código. E lembre-se: ao criar um pacote, você também aplica tudo que vimos de dependências – declare no pacote seu `install_requires` adequadamente, use `semver` para versionar seu pacote e aproveite `Poetry`/`UV` se quiser agilizar o processo (ambos geram estrutura de pacote e facilitam publicação). É literalmente colocar em prática engenharia de software em ciência de dados!

MERCADO, CASES E TENDÊNCIAS

Tendências em Ciência de Dados para 2025

A análise de dados segue em rápida evolução, impulsionada por IA generativa, automação de pipelines e descentralização via Data Mesh. O artigo de Gabriel Beran Ribeiro destaca a ascensão do Data Engineering, a integração de GenAI em plataformas de BI e o crescimento do Edge Computing para análises em tempo real. Essas tendências exigem profissionais cada vez mais preparados(as) para lidar com ambientes complexos e dependências bem gerenciadas. Leia o artigo completo [aqui](#).

Desafios de Dados e Analytics segundo Gartner

O Gartner elenca nove desafios para 2025, incluindo produtos de dados altamente consumíveis, gerenciamento de metadados e data fabric multimodal. Destaca-se a importância de produtos de dados reutilizáveis e a automação de resultados via agentes de IA, reforçando a necessidade de ambientes reproduzíveis e dependências bem documentadas. Veja os desafios [aqui](#).

Abordagem Bibliométrica sobre Gestão de Dados

Estudo bibliométrico revela crescimento acelerado nas publicações sobre gestão de dados, destacando a influência de autores e periódicos voltados à tecnologia e bancos de dados. Leia o estudo [aqui](#).

Curso de Fundamentos da Ciência de Dados – FM2S

Vídeo introdutório sobre como transformar dados em decisões estratégicas, com ênfase em ambientes organizados e dependências bem gerenciadas. Assista no [YouTube](#).

Podcast “Pizza de Dados”

O primeiro podcast brasileiro sobre ciência de dados, com episódios sobre arquitetura de dados, data lakes, governança e qualidade de dados. Ouça [aqui](#).

Lista de Podcasts Relevantes

A DataCamp compilou os melhores podcasts sobre ciência de dados, abordando temas como IA, machine learning, governança e tendências do mercado. Confira a lista [aqui](#).

EMPRE

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos como gerenciar dependências de forma eficaz em projetos de ciência de dados – um elemento-chave para garantir reprodutibilidade e evitar surpresas desagradáveis. Você aprendeu a importância de isolar ambientes por projeto, utilizando ferramentas como `virtualenv/venv` ou `Conda` para prevenir conflitos entre bibliotecas.

Viu também como documentar e fixar requisitos em arquivos de requirements ou lockfiles, assegurando que todos (inclusive você no futuro!) possam recriar o mesmo ambiente de desenvolvimento e execução. Discutimos noções de versionamento semântico e melhores práticas ao especificar versões, equilibrando flexibilidade e estabilidade.

Abordamos as ferramentas modernas (`Poetry`, `Pipenv`, `UV`) que simplificam o gerenciamento de dependências e automatizam tarefas, além de compararmos suas vantagens. Em seguida, você conheceu técnicas de empacotamento de código reutilizável, aprendendo que transformar funções comuns em pacotes compartilháveis aumenta a modularidade e facilita a manutenção de projetos maiores.

Por fim, examinamos casos do mundo real e tendências: a crescente cultura DevOps/MLOps aplicada a data science, o impacto da gestão de dependências na colaboração e alguns alertas de casos famosos. Com isso, esperamos que você esteja mais preparado(a) para estruturar seus futuros projetos de maneira robusta, unindo o melhor da engenharia de software com as necessidades da ciência de dados.

Lembre-se: um ambiente bem gerenciado é a base para que suas ideias e análises brilhem, sem serem atrapalhadas por questões técnicas de versão ou compatibilidade. Boas práticas hoje economizam tempo amanhã – e permitem que você foque no que realmente importa: extrair valor dos dados com confiança e consistência!

REFERÊNCIAS

IBM DATA SCIENCE COMMUNITY. **Dependency Management – Best Practices**. 2019. Disponível em: <https://github.com/IBM/data-science-best-practices/blob/main/dependency_management.md>. Acesso em: 15 dez. 2025.14 nov. 2025.

KILLI, J. **What every data scientist should know about Python dependencies**. 2022. Disponível em: <<https://valohai.com/blog/dependency-management-for-data-science/>>. Acesso em: 14 nov. 2025.15 dez. 2025.

LARANJA, Emerson. **Versionamento Semântico (SemVer): uma breve introdução**. 2022. Disponível em: <<https://www.alura.com.br/artigos/versionamento-semantico-breve-introducao>>. Acesso em: 15 dez. 2025.14 nov. 2025.

MEHER, S. **A Beginner's Guide to Packaging Python Code**. 2025. Disponível em: <https://shibu778.github.io/posts/packaging_python/>. Acesso em: 15 dez. 2025.14 nov. 2025.

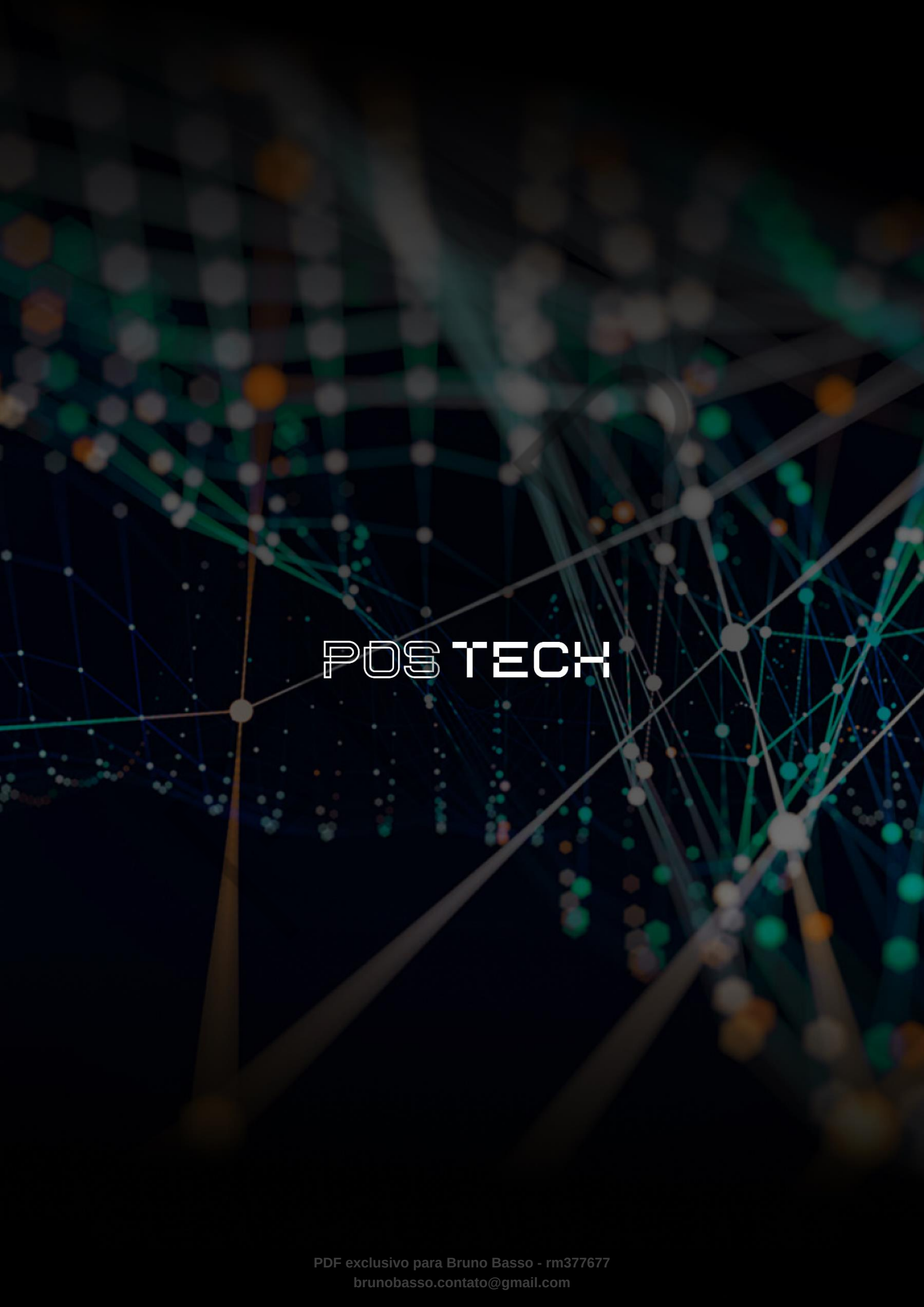
TREADWAY, A. **Software Engineering for Data Scientists**. Manning (MEAP), 2024. Disponível em: <<https://www.manning.com/books/software-engineering-for-data-scientists>>. Acesso em: 15 dez. 2025.14 nov. 2025.

ULILI, S. **A Deep Dive into UV: The Fast Python Package Manager**. 2025. Disponível em: <<https://betterstack.com/community/guides/scaling-python/uv-explained/>>. Acesso em: 15 dez. 2025.14 nov. 2025.

PALAVRAS-CHAVE

Gerenciamento de dependências. Reprodutibilidade em ciência de dados.
Ambientes virtuais e versionamento.

EMANIP



POSTECH